

Homework 1

Student ID: 706709610

Problem 1.

- (a) Prove by induction on n that for all integers $n \geq 1$, $1 + 4 + 7 + \dots + (3n - 2) = n(3n - 1)/2$.
- (b) Let x and y be positive real numbers. Prove by contradiction: if $x^2 - y^2 = 1$, then x or y (or both) are not integers.

Proof of (a). We first prove the base case, i.e., $n = 1$. When $n = 1$, the LHS is 1. The RHS is $(1)(3(1) - 1)/2 = 1$.

Now assume we have $1 + 4 + 7 + \dots + (3n - 2) = n(3n - 1)/2$ up to some $n \geq 1$. Then

$$\begin{aligned} 1 + 4 + 7 + \dots + (3n - 2) + (3(n + 1) - 2) &= \frac{n(3n - 1)}{2} + 3(n + 1) - 2 \\ &= \frac{n(3n - 1) + 6(n + 1) - 2}{2} \\ &= \frac{3n^2 - n + 6n + 6 - 2}{2} \\ &= \frac{3n^2 + 5n + 4}{2} \\ &= \frac{(n + 1)(3n + 4)}{2} \\ &= \frac{(n + 1)(3(n + 1) - 1)}{2}. \end{aligned}$$

By induction on n we are done. □

Proof of (b). Let x and y be positive real numbers such that $x^2 - y^2 = 1$. Suppose for the sake of contradiction that $x, y \in \mathbb{Z}$. By difference of squares formula we have $x^2 - y^2 = (x - y)(x + y)$. Then by assumption we have $(x - y)(x + y) = 1$. Since $x, y \in \mathbb{Z}$ then $x - y, x + y \in \mathbb{Z}$. We have two cases, $x - y = 1$ and $x + y = 1$ or $x - y = -1$ and $x + y = -1$.

Case 1: $x - y = 1$ and $x + y = 1$ means that $x = 1$ and $y = 0$. This is a contradiction since x and y must be both positive.

Case 2: $x - y = -1$ and $x + y = -1$ means that $x = -1$ and $y = 0$. This is a contradiction since both x and y must be positive.

Since we have covered all cases and have a contradiction in both, then it follows that our assumption must be false and we have that x or y (or both) are not integers. □

Problem 2. Indicate the relationships between each pair of functions (A, B) listed below.

Solution:

A	B	$A \in o(B)$	$A \in O(B)$	$A \in \theta(B)$	$A \in \Omega(B)$	$A \in \omega(B)$
$\log_2(n)$	$\log_5(n)$	No	Yes	Yes	Yes	No
$\log \log(n)$	$\sqrt{\log(n)}$	Yes	Yes	No	No	No
$n^{3/4} \log(n)$	$n^{\log(n)}$	Yes	Yes	No	No	No
$2^{\log^6(n)}$	n^6	No	No	No	Yes	Yes
$n^3 2^n$	3^n	Yes	Yes	No	No	No

□

Problem 3. For each of the pseudocode snippets below, give the asymptotic running time in Θ notation and explain how you derived it.

```
(a) for i = 1 to n do
    j = 0;
    while j <= i do
        j = j + 2;
    end
end
```

```
(b) i = 2;
while i <= n do
    i = i^2;
end
```

Solution:

(a) Claim: the algorithm is $\Theta(n^2)$.

For fixed i , the inner loop does $\lfloor i/2 \rfloor + 1$ iterations. I figured this out through trial and error of different i values. Since this happens for each iteration of the outer loop and the outer loop does n iterations, then we have $\sum_{i=1}^n (\lfloor i/2 \rfloor + 1)$ total iterations. The floor function only adds a constant factor so we can get rid of it. We will have $\Theta(\sum_{i=1}^n i + 1) = \Theta(n(n+1)/2 + n) = \Theta(n^2)$.

(b) Claim: the algorithm is $\Theta(\log \log(n))$.

We can trace through the while loop and see what happens to i after each iteration. We get: $i_0 = 2$, $i_1 = 2^2$, $i_2 = 2^4$, $i_3 = 2^8$, ..., $i_n = 2^{2^n}$. So the stopping point is $\log \log(n)$ iterations, i.e., the algorithm is $\Theta(\log \log(n))$ as we claimed.

□

Problem 4. Let A be an array of size n which contains only positive integer. Give an algorithm which takes as input such an array A and finds the SMALLEST element with MINIMUM frequency. Additionally, provide the time complexity of your algorithm in terms of n .

Solution:

```
input: Let A be the array of size n

let F be an empty hash-map/dictionary

// set all frequencies
for item in A do
    F[item] = F[item] + 1;
end

//get min freq
let min_freq = n;
for item in A do
    if F[item] < min_freq do
        min_freq = F[item];
    end
end

//get all elements with min frequency
let M be an empty list;
for item in A do
    if F[item] == min_freq do
        add item to M;
    end
end

//find min value among min frequencies
let num be the first element of M;
for item in M do
    if item < num do
        num = item;
    end
end

return num;
```

Complexity Analysis: The first loop we iterate over all n elements on A and perform some arithmetic operation. This loop is $O(n)$. The next loop we also iterate over every element of A and perform $O(1)$ operations. So this loop is also $O(n)$. The next loop we again loop over A and perform $O(1)$ operations. This loop is $O(n)$. The last loop we iterate over M which is at most of size n so the loop is $O(n)$ since it only contains $O(1)$ operations. So in total our algorithm is $O(n)$. We must note that this complexity analysis assumes that hash-map/dictionary operations are $O(1)$ which is expected, but at worst they can be $O(n)$. In that case our algorithm would be $O(n^2)$. $\square$

Problem 5. *Show that there is always a stable assignment of students to hospitals, and give an algorithm to find one. Additionally provide a time complexity analysis of the algorithm.*

Solution:

```
Algorithm(Preference list for n Students and m Hospitals)
  let M be an empty map;
  while (some student s is unmatched and hasn't proposed to every hospital) do
    let h be the first hospital on s's preference list which they haven't proposed to;
    if (M[h] isn't full) add s to h's matches in M;
    else
      let s' be h's least preferred student;
      if (h prefers s to s') remove s' from M[h] and replace add s to M[h];
      else h rejects s, M stays the same
  return M;
```

We now prove that this algorithm always finds a stable matching.

Proof. Notice the following observation: if a hospital h rejects a student s at any point during the algorithm, then h will prefer every student in its final matching to s . This is because h will always bump it's worst student in favor of a better student which is proposing. So if s got bumped from h , it's because s was the worst student and a proposing student was better. If s proposed to h and got rejected, it's because h didn't prefer s over it's worst student at the time. Since h 's students only get better then s must be worse than the final selection.

Assume for the sake of contradiction that algorithm outputs a matching M which is not stable, i.e., it has at least one instability. We have two cases.

Case 1: There is a student s and hospital h with $s \in M[h]$ and some s' not assigned to any hospital such that h prefers s' to s . Then since the algorithm is over, s' must've proposed to h at some point and got rejected at some point (either immediately or later bumped). By the observation, since s' was rejected then all students in $M[h]$ are better than s' . Contradiction.

Case 2: There are students s and s' and hospitals h and h' such that $s \in M[h]$ and $s' \in M[h']$ but h prefers s' to s and s' prefers h to h' . We have two subcases:

- s' never proposed to h . Note that s' makes proposals in decreasing order of preference, so h' must've been higher on s' preference list than h . Contradiction.
- s' did propose to h at some point. This must mean that h rejected s' at some point. By our observation, h 's final selection must be preferred over s' . Contradiction.

□

Complexity Analysis: We claim the algorithm is $O(n^2)$. In the while loop, every operation is $O(1)$ besides finding the worst student in some hospital h 's preference list. Each hospital has a set amount

of space allocated for students, call this c_h . Then at most n students can propose to h . So at most h has to look up its worst student n times, which would come out to $O(c_h \cdot n)$. Since there are m hospitals, then in total the algorithm would be $O(\sum_{h=1}^m c_h \cdot n) \leq O(n^2)$ since $\sum_{h=1}^m c_h \leq n$. This is because we take the assumption that there are less hospital spots than students. $\square$

Problem 6. *Show that such a set of truncations can always be found, and give an algorithm to find them. Additionally, provide a time complexity analysis of your algorithm.*

Solution: We first give an algorithm and then prove that it works.

```

Algorithm(a map S of the schedule of each ship)
  initialize an empty map M;
  while (a ship s is unmatched and hasn't proposed to all ports on its schedule) do
    let p be the first port on s's schedule which it hasn't proposed to;
    if (p is unmatched) add (s, p) to M;
    else
      let s' be the ship which p is paired with;
      if (p is sooner on s' schedule than s) remove (s', p) from M and add (s,p) to M;
      else leave M the same;
  return M;

```

We now will prove that the algorithm works and always produces a set of truncations which satisfies that no two ships will ever be at the same port on the same day.

Proof. We will establish some notation before starting. For each ship s matched to some port p_s let $D(s)$ be the day that s visits p_s in its original schedule. We need to show that for every day t , no two ships are at the same port. Fix some day t . Assume for the sake of contradiction that there exists two ships s_1 and s_2 which are at the same port p on day t . We have 3 cases:

Case 1: Both ships are not at their matched port yet. This means they are still following their original schedule which we assume makes it so no two ships are at the same port. Contradiction

Case 2: Both ships are stationed at their assigned port. Notice that our algorithm will never have two ships matched to the same port. This is because in the case where a ship s proposes to a matched port p and we add (s,p) to M , then we also remove the matching between the other ship (s',p) from M . Contradiction.

Case 3: One ship, WLOG s_1 , is stationed at its matched port p_{s_1} since $D(s_1) \leq t$, and s_2 is still not at its matched port on day t . But since we are assuming that s_2 is at port p_{s_1} on day t , then by Case 1, we must have that $D(s_1) < t$ because two ships can't be at the same port on the same day in their original schedule. Then since the proposal order of each ship is the same as their schedule and we have $D(s_1) < t < D(s_2)$, then at some point s_2 proposed to p_{s_1} . Then either s_2 was rejected immediately or later bumped. Our algorithm is analogous to Gale-Shapely. So the observation from before still applies,

i.e., if port p rejects ship s , then p 's final match will arrive later than ship s . So this means that s_2 must arrive at port p_{s_1} before s_1 is stationed there. Contradiction. $\square$

Complexity Analysis: The analysis here is the same as Gale-Shapely. We claim the algorithm is $O(n^2)$ where n is the number of ships and ports. Each ship will propose to at most n ports and since there are n ports, there is at most n^2 iterations of the while loop. Each operation in the while loop is $O(1)$, so it follows that the algorithm is $O(n^2)$. $\square$

Problem 7. *Give a proof that, for any set of preference lists, switching the order of a pair on the list cannot improve a woman's partner in the Gale-Shapley algorithm*

Proof. Let W be w 's true preference list and W' be the preference list with m and m' swapped which rank lower than her true partner m^* . Suppose we run Gale-Shapely twice, one with W , where w ends up with w^* and the other with W' , where w ends up with some other man m'' who ranks higher than m^* . Notice that m^* 's behaviour is completely unaffected by w 's preference list, so m^* will still propose to w at the same moment in both W and W' runs. When m^* does propose, w is either holding no one or some man m . If w is holding no one, then w will accept m^* . If not w will compare m^* to m . Notice that since the only difference between the W and W' runs is the swap between lower ranked m and m' , then the only difference that could have occurred is that w holds m instead of m' , vice versa, or someone who is different than m and m' but still ranked below m^* . Since in all cases m^* is ranked highest, then w will accept m^* . Notice that once w accepts m^* , it will stay like that since now the remainder of the L' run is the same as the L run. Thus w will end up with m^* in both runs. Contradiction. $\square$